DAY 26

n8n + External APIs

REST Integration, Authentication, Pagination, Rate Limits, Reliability, and Production API Workflows

COURSE
GPT + AI Agents + RAG + n8n + API Integration + Business Automation

LECTURE FORMAT
60-minute detailed lecture | Beginner to Intermediate | Hands-on business automation

CAPSTONE FOR TODAY
Build a Multi-API Customer Operations Workflow that retrieves trusted live data, calls an AI layer only for interpretation, applies deterministic rules, and writes safe results back to external systems.

Week 4 - n8n + APIs + Business Automation

Day 25: n8n + GPT -> Day 26: n8n + External APIs -> Day 27: Business Automation Patterns

Table of Contents

Table of Contents - Continued

NAVIGATION NOTE

Use the PDF outline/bookmarks panel to jump directly to lecture sections.

Day 26 Learning Objectives

- Explain what an external API is and how n8n communicates with it.
- Design REST requests using method, URL, headers, query parameters, path parameters, and body data.
- Choose the right authentication pattern: API key, bearer token, Basic Auth, OAuth2, or predefined n8n credentials.
- Use the n8n HTTP Request node for services that do not have a dedicated n8n node.
- Map data between multiple APIs without losing IDs, timestamps, or business context.
- Handle pagination, rate limits, retries, timeouts, and transient failures.
- Distinguish safe read operations from risky write operations.
- Validate response schemas and business rules before downstream actions.
- Protect secrets and prevent accidental credential leakage into AI prompts or logs.
- Build reusable API connector sub-workflows and a production-ready multi-API automation.

60-Minute Lecture Schedule

Time	Topic
0-5 min	Review Day 25 and define the API integration problem
5-12 min	REST/HTTP request anatomy
12-20 min	Authentication and credentials
20-29 min	n8n HTTP Request node in practice
29-36 min	Mapping, pagination, and multi-item processing
36-43 min	Rate limits, retries, errors, and idempotency
43-49 min	Security and production design
49-56 min	Mini project: Multi-API Customer Operations Workflow
56-60 min	Quiz, homework, and Day 27 preview

KEY TAKEAWAY

Day 26 is not about memorizing one vendor API. It is about learning a repeatable integration method that works across CRMs, ecommerce platforms, ticketing systems, shipping providers, finance tools, and internal services.

1. From AI Workflow to Connected Business System

DAY 25

n8n -> GPT -> structured interpretation -> business rules

DAY 26

n8n -> external APIs -> live business data -> safe updates -> reliable automation

```
USER / EVENT
     |
     v
    n8n
     |
     +--> CRM API ---------> customer / lead facts
     |
     +--> Order API -------> current order facts
     |
     +--> Tracking API ----> shipment facts
     |
     +--> GPT -------------> classify / summarize / draft
     |
     +--> Ticket API ------> create or update action
     v
AUDIT LOG + FINAL RESULT
```

Why APIs Matter for AI Automation

- **GPT knows language;** business APIs know current operational facts.
- **n8n orchestrates;** APIs provide read and write capabilities.
- **RAG retrieves knowledge;** APIs retrieve live records and perform transactions.
- **AI can recommend;** business rules and APIs determine what is actually allowed and executed.

KEY TAKEAWAY

A production AI automation system is usually an integration system with AI inside it - not an AI prompt with a few integrations attached.

2. What Is an External API?

An API (Application Programming Interface) is a defined way for software systems to exchange requests and responses. In most business automation projects, you will interact with HTTP-based REST APIs.

Client-Server Mental Model

```
n8n (CLIENT)                      BUSINESS SERVICE (SERVER)
     |                                        |
     |  HTTP REQUEST                          |
     |--------------------------------------->|
     |                                        |
     |  HTTP RESPONSE                         |
     |<---------------------------------------|
     |                                        |
   workflow continues using returned data
```

Typical Business APIs

System	Read Examples	Write Examples
CRM	lead, contact, company, opportunity	create lead, update stage, add note
Ecommerce	order, customer, product, inventory	update fulfillment, tags, notes
Support	ticket, conversation, SLA	create ticket, assign, add reply
Shipping	tracking, carrier events, ETA	create shipment, cancel label
Finance	invoice, payment, balance	create invoice, record payment
Internal API	customer profile, risk data, project status	update workflow state

REST Resource Thinking

```
GET    /customers/123        -> read customer
GET    /orders?customer=123 -> search orders
POST   /tickets              -> create ticket
PATCH  /tickets/555          -> update part of ticket
PUT    /profiles/123         -> replace/update resource
DELETE /drafts/987           -> delete resource
```

3. Anatomy of an HTTP API Request

Part	Purpose	Example
Method	What type of operation	GET, POST, PATCH, DELETE
URL	Where the resource lives	https://api.example.com/orders/1002
Path parameter	Identifies resource inside URL	/orders/1002
Query parameter	Filters/sorts/paginates	?status=open&limit=50
Header	Metadata/auth/content type	Authorization: Bearer ...
Body	Data sent to server	JSON object for create/update
Timeout	How long caller waits	30 seconds
Response	Status + headers + body	200 + JSON

Example GET Request

```
GET /v1/orders/ORD-1002 HTTP/1.1
Host: api.example.com
Authorization: Bearer <TOKEN>
Accept: application/json
```

Example POST Request

```
POST /v1/tickets HTTP/1.1
Host: api.example.com
Authorization: Bearer <TOKEN>
Content-Type: application/json

{
  "customer_id": "C1002",
  "subject": "Delayed shipment",
  "priority": "high"
}
```

Response Example

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "TKT-5501",
  "status": "open",
  "priority": "high"
}
```

4. HTTP Methods and Safe Automation Design

Method	Typical Meaning	Usually Side Effect?	Automation Risk
GET	Read resource	No	Low
POST	Create / execute	Yes	Medium-High
PUT	Replace/update resource	Yes	High
PATCH	Partial update	Yes	Medium-High
DELETE	Delete resource	Yes	High

Read Before Write Pattern

```
1. GET current record
2. Validate ownership / state / eligibility
3. Decide desired change
4. Apply deterministic business rules
5. Human approval if sensitive
6. PATCH / POST / DELETE
7. Re-read or validate response
8. Log final outcome
```

Idempotency

Idempotency means designing a write so that retrying the same logical request does not accidentally create duplicate business actions.

EXAMPLE

If an API supports an idempotency key, send a unique stable key for the business event, such as order-delay:ORD-1002:2026-08-19. If the workflow retries, the server can recognize the same request instead of creating a second ticket.

KEY TAKEAWAY

Retries are safe only when you understand the side effects. Retrying a GET is usually simpler than retrying POST /payments or POST /refunds.

5. Status Codes: Read the API Result Correctly

Code Family	Meaning	Typical Workflow Decision
2xx	Success	Validate response and continue
3xx	Redirect	Usually handled by HTTP client; verify if unusual
400	Bad request	Fix payload/parameters; do not blind retry
401	Unauthenticated	Credential/token problem
403	Authenticated but forbidden	Permission/scope problem
404	Resource not found	Ask user, branch, or create if appropriate
409	Conflict	Duplicate/state conflict; inspect before retry
422	Validation failure	Correct input/business data
429	Rate limited	Wait/backoff then retry within limits
5xx	Server-side failure	Bounded retry then fallback/escalate

Error Classification

```
ERROR
  |
  +-- Input / validation? --------> fix data or ask user
  +-- Authentication? -----------> refresh/fix credential
  +-- Authorization? ------------> stop / escalate
  +-- Not found? ----------------> branch / ask / fallback
  +-- Rate limit? ---------------> wait + retry
  +-- Temporary server error? ---> bounded retry
  +-- Permanent business rule? --> stop safely
```

6. Authentication Patterns

Pattern	How It Works	Common Use
API key	Secret key in header/query	SaaS APIs and internal services
Bearer token	Authorization: Bearer token	OAuth access tokens / personal tokens
Basic Auth	username + password encoded by client	Legacy/internal APIs
OAuth2	User/app grants scoped access; tokens can refresh	Google, Microsoft, many SaaS apps
Custom header	Vendor-specific secret/signature	Private/vendor APIs
mTLS / signed requests	Cryptographic client identity/signing	High-security financial/internal APIs

n8n Credential Principle

Store authentication in n8n credentials rather than hard-coding secrets in node parameters or expressions. Current n8n documentation describes credentials as securely stored authentication information used to connect workflows to external APIs and services.

- Use predefined credentials when n8n supports the service.
- Use generic HTTP Request credentials for APIs without a dedicated node.
- Give credentials only the minimum scopes required.
- Separate development and production credentials.
- Rotate leaked or old tokens.
- Never send API keys into GPT prompts, chat messages, or ordinary business logs.

7. n8n HTTP Request Node - Core Configuration

The HTTP Request node is n8n's general-purpose connector for REST APIs. It can call services even when there is no dedicated n8n integration node.

Field	What You Configure
Method	GET, POST, PUT, PATCH, DELETE, and supported methods
URL	Static endpoint or dynamic n8n expression
Authentication	Credential type / predefined or generic auth
Query Parameters	Filters, pagination, search options
Headers	Content-Type, Accept, vendor headers, correlation IDs
Body	JSON, form data, raw payload depending on API
Response	Data format and output handling
Options	Timeouts, redirects, response details, batching/pagination depending on use case
Settings	Retry On Fail / On Error behavior at the node level

Dynamic URL Example

```
https://api.example.com/v1/orders/{{$json.order_id}}
```

Dynamic Query Example

```
customer_id = {{$json.customer_id}}
status      = open
limit       = 50
```

Dynamic JSON Body

```
{
  "customer_id": "{{$json.customer_id}}",
  "summary": "{{$json.ai.summary}}",
  "priority": "{{$json.priority}}",
  "source": "n8n"
}
```

8. From API Documentation to n8n Node

1. Find the exact endpoint and HTTP method in the API documentation.
2. Identify authentication requirements and required scopes.
3. List required path, query, header, and body fields.
4. Build the smallest successful request first.
5. Test the request with known data.
6. Inspect status code and response JSON.
7. Map response data into a normalized internal structure.
8. Add error branches, retries, rate-limit handling, and logging.
9. Only after read operations are stable, add write operations.
10. Test with duplicate events and failure cases before production.

Documentation Reading Checklist

Question	Example
Base URL?	https://api.vendor.com/v1
Endpoint?	GET /orders/{id}
Auth?	Bearer token
Required scopes?	orders:read
Pagination?	cursor or page/limit
Rate limit?	100 requests/minute
Error format?	{"error":{"code":"..."}}
Idempotency?	Idempotency-Key header
Webhook support?	order.updated
Sandbox?	separate test base URL

9. Normalizing External API Data

Different APIs name the same business concept differently. A professional workflow should normalize vendor-specific responses into an internal schema before AI or business rules use them.

Vendor A	Vendor B	Internal Field
customerId	customer_id	customer_id
orderNumber	id	order_id
shipState	tracking_status	shipping_status
emailAddress	email	email
createdAt	created_at	created_at

Normalize With Set/Edit Fields or Code

```
{
  "customer_id": "{{$json.customerId}}",
  "order_id": "{{$json.orderNumber}}",
  "shipping_status": "{{$json.shipState}}",
  "source_system": "vendor_a"
}
```

KEY TAKEAWAY

Normalize early. AI prompts, routing logic, databases, and reports become easier when downstream nodes work with one stable internal schema.

10. Mapping IDs Across Systems

One customer can have different identifiers in different platforms. Never assume IDs are interchangeable.

```
CRM customer_id:       CRM-7281
Ecommerce customer_id: SHOP-9912
Support requester_id:  ZD-4100
Internal user_id:      U-1002
```

Cross-System Mapping Table

internal_user_id	crm_id	commerce_id	support_id
U-1002	CRM-7281	SHOP-9912	ZD-4100
U-1003	CRM-7298	SHOP-9933	ZD-4114

Good Integration Rule

DO

Carry each system-specific ID explicitly. Use a mapping table, database, or trusted lookup service.

DO NOT

Ask the LLM to guess which IDs correspond to the same customer.

11. Pagination - Getting More Than One Page

Many APIs return only a limited number of records per request. Pagination is the process of repeatedly requesting the next result page until the desired data is collected.

Common Pagination Styles

Style	Example	How Next Page Is Found
Page number	?page=3&limit=100	increment page
Offset	?offset=200&limit=100	offset += limit
Cursor	?cursor=eyJ...	use next_cursor from response
Link URL	response.next_url	request returned URL
Token	nextPageToken	use token returned by API

Cursor Pagination Example

```
First response:
{
  "data": [...],
  "next_cursor": "abc123"
}

Next request:
GET /contacts?cursor=abc123&limit=100
```

Pagination Stop Conditions

- No next cursor/token exists.
- Returned list is empty.
- Returned count is less than page size for page/offset styles.
- A configured maximum page/item limit is reached.
- Business date/time boundary is reached.

12. Pagination in n8n

The n8n HTTP Request node includes built-in pagination capabilities for APIs that return data in multiple pages. The exact pagination expression depends on the vendor's scheme.

```
REQUEST 1
GET /contacts?limit=100
       |
       v
response.next_cursor = abc123
       |
       v
REQUEST 2
GET /contacts?limit=100&cursor=abc123
       |
       v
continue until next_cursor is empty
```

Production Pagination Guardrails

- Maximum page count to prevent infinite pagination.
- Maximum item count to control memory and cost.
- Date filters so you fetch only new/changed records.
- Checkpoint last successful cursor/time for large sync jobs.
- Rate-limit awareness between requests.
- Logging of page number/cursor and record count.

KEY TAKEAWAY

Do not download an entire CRM every five minutes when the API supports updated_since, cursor checkpoints, or webhooks.

13. Rate Limits and Backoff

External APIs protect themselves with rate limits. Current n8n documentation notes that a node hitting a rate limit errors and recommends patterns such as Retry On Fail and loop/batch workflows depending on the API and data volume.

Common Signals

Signal	Meaning	Response
HTTP 429	Too many requests	wait then retry
Retry-After header	server specifies wait duration	honor it when possible
X-RateLimit-Remaining	requests remaining	throttle proactively
X-RateLimit-Reset	reset timestamp	delay until reset

Exponential Backoff Concept

```
attempt 1 fails -> wait 1 second
attempt 2 fails -> wait 2 seconds
attempt 3 fails -> wait 4 seconds
attempt 4 fails -> stop / queue / escalate
```

Rate Limit Design

- Limit concurrency.
- Process records in batches.
- Use Wait node when required.
- Use Retry On Fail with sensible wait time and attempt count.
- Respect vendor-specific limits.
- Prefer webhook/event updates over aggressive polling when available.

14. Retry Strategy

Failure	Retry?	Why
Timeout	Usually yes, bounded	May be temporary
429 rate limit	Yes after delay	Expected transient limit
500/502/503	Usually yes, bounded	Server may recover
400 invalid JSON	No	Request must be fixed
401 bad token	Not blindly	Credential needs refresh/fix
403 forbidden	No	Permissions/scopes must change
404 missing record	Usually no	Business path should handle missing data
409 conflict	Inspect first	Retry may duplicate or repeat conflict

Retry + Idempotency

```
IF request is read-only:
    bounded retry is often safe

IF request creates money / tickets / orders:
    use idempotency key or duplicate check
    BEFORE retrying
```

IMPORTANT

"Retry On Fail" is a reliability mechanism, not a substitute for understanding the API operation. A POST that creates a resource may succeed on the server even if the client times out before receiving the response.

15. n8n Error Handling Patterns

```
HTTP Request
     |
     +-- Success ----------> Validate -> Continue
     |
     +-- Recoverable ------> Retry / Wait
     |
     +-- Business error ---> Branch / Ask / Queue
     |
     +-- Fatal ------------> Error Workflow / Alert
```

Useful n8n Controls

Control	Purpose
Retry On Fail	Retry node after failure using configured attempts/wait
On Error behavior	Stop workflow or continue via chosen behavior
Error Trigger workflow	Centralized handling for failed executions
Stop And Error node	Intentionally fail when business validation fails
IF/Switch	Branch based on status/error payload
Wait	Delay for backoff or external timing
Loop Over Items	Process batches instead of large concurrent bursts

Structured Error Object

```
{
  "success": false,
  "system": "tracking_api",
  "operation": "get_tracking",
  "status_code": 503,
  "error_code": "SERVICE_UNAVAILABLE",
  "retryable": true,
  "attempt": 2
}
```

16. Timeouts and Slow APIs

- Set realistic request timeouts rather than waiting indefinitely.
- Do not use synchronous webhook responses for long-running chains when a queue/background workflow is more appropriate.
- Cache stable reference data when permitted.
- Parallelize only independent calls and respect vendor rate limits.
- Use a circuit-breaker-like policy: after repeated failures, stop calling and route to fallback/manual processing.

Latency Budget Example

Step	Approx. Budget
CRM lookup	0.5-1.0 s
Order lookup	0.5-1.0 s
Tracking lookup	0.5-2.0 s
GPT interpretation	1-4 s
Ticket write	0.5-1.0 s

KEY TAKEAWAY

The fastest API call is the unnecessary call you eliminated. Design workflows to retrieve only the data required for the current business decision.

17. Webhooks vs Polling vs Scheduled API Calls

Pattern	Best When	Tradeoff
Webhook	Provider emits events in real time	Needs secure receiver and duplicate handling
Polling	No webhook exists	Can waste API calls and increase latency
Scheduled delta sync	Periodic reporting/reconciliation	Not real-time but controllable
On-demand API	User/workflow asks for current fact	Best for interactive live lookup

Hybrid Pattern

```
Webhook says: order.updated
        |
        v
n8n receives lightweight event
        |
        v
GET /orders/{id}
        |
        v
retrieve authoritative current record
        |
        v
process safely
```

This pattern is common because webhook payloads may be intentionally small or may represent an event rather than the full authoritative object.

18. API Security for Automation Engineers

Risk	Bad Practice	Better Practice
Secret leakage	Put token in Code node text	Use n8n credentials
Over-permission	Admin token for read task	Least-privilege scopes
Data exposure	Send full customer record to GPT	Send only needed fields
IDOR / access bypass	Trust user-supplied customer ID	Use authenticated identity + server-side ownership check
Logging secrets	Log headers/body indiscriminately	Redact sensitive fields
Write abuse	Let model call unrestricted write API	Guardrails + business rules + approval
SSRF/untrusted URL	Let user choose arbitrary request URL	Allowlist endpoints / fixed base URLs

Separate Trusted and Untrusted Data

```
TRUSTED
- authenticated user ID
- API credential
- permission scopes
- business rule thresholds

UNTRUSTED
- user text
- webhook content
- external document text
- model output until validated
```

19. Validation: API Success Is Not Business Success

A 200 response means the HTTP operation succeeded. It does not prove the returned data is complete, allowed, fresh, or suitable for the next action.

Three Validation Layers

```
LAYER 1 - TRANSPORT
HTTP success? valid JSON?

LAYER 2 - SCHEMA
Required fields? correct types?

LAYER 3 - BUSINESS RULES
Correct customer? current state? action allowed?
```

Example

```
API returns 200:
{
  "order_id": "ORD-1002",
  "status": "cancelled",
  "tracking_id": null
}

Business rule:
Do NOT call tracking API because cancelled orders have no active shipment.
```

20. AI + External API: Proper Responsibility Split

Responsibility	Best Component
Understand free-text request	GPT/AI
Authenticate user	Application/auth provider
Retrieve order	Order API
Calculate amount > threshold	Code/n8n IF
Retrieve policy	RAG/knowledge base
Decide semantic category	AI
Authorize refund	Business workflow/human
Execute refund	Payment API after approval
Generate friendly response	AI

Reliable Pattern

```
USER MESSAGE
     |
     v
GPT extracts intent + identifiers
     |
     v
VALIDATE STRUCTURE
     |
     v
EXTERNAL API retrieves live facts
     |
     v
BUSINESS RULES decide allowed path
     |
     v
GPT drafts human-friendly response
     |
     v
OPTIONAL WRITE API after approval
```

KEY TAKEAWAY

Never ask the model to invent the result of an external API call. If current data is required, call the authoritative system.

21. Reusable API Connector Sub-Workflows

As n8n projects grow, repeated HTTP logic should be packaged into reusable sub-workflows.

Example Connector Contract

```
INPUT
{
  "order_id": "ORD-1002"
}

SUB-WORKFLOW
- validates order_id
- calls Order API
- normalizes response
- converts errors to standard structure

OUTPUT
{
  "success": true,
  "data": {
    "order_id": "ORD-1002",
    "status": "shipped"
  },
  "error": null
}
```

Benefits

- One place to update authentication or endpoint versions.
- Consistent error handling.
- Consistent normalized output.
- Cleaner business workflows.
- Easier testing and observability.

22. Multi-API Orchestration Patterns

Pattern A - Enrichment

```
Lead
 -> CRM lookup
 -> Company data API
 -> AI summary
 -> CRM update
```

Pattern B - Reconciliation

```
Schedule
 -> Finance API invoices
 -> Payment API transactions
 -> Match IDs / amounts
 -> Exception report
```

Pattern C - Customer Support

```
Ticket
 -> Customer API
 -> Order API
 -> Tracking API
 -> Policy RAG
 -> AI draft
 -> Support API update
```

Pattern D - Inventory

```
Order event
 -> Product API
 -> Inventory API
 -> threshold rules
 -> Supplier API / alert
```

23. Mini Project - Multi-API Customer Operations Workflow

BUSINESS PROBLEM

Support agents receive shipping-delay messages. They waste time switching between customer, order, tracking, policy, and ticket systems. Build one n8n workflow that assembles verified context and safely updates the support ticket.

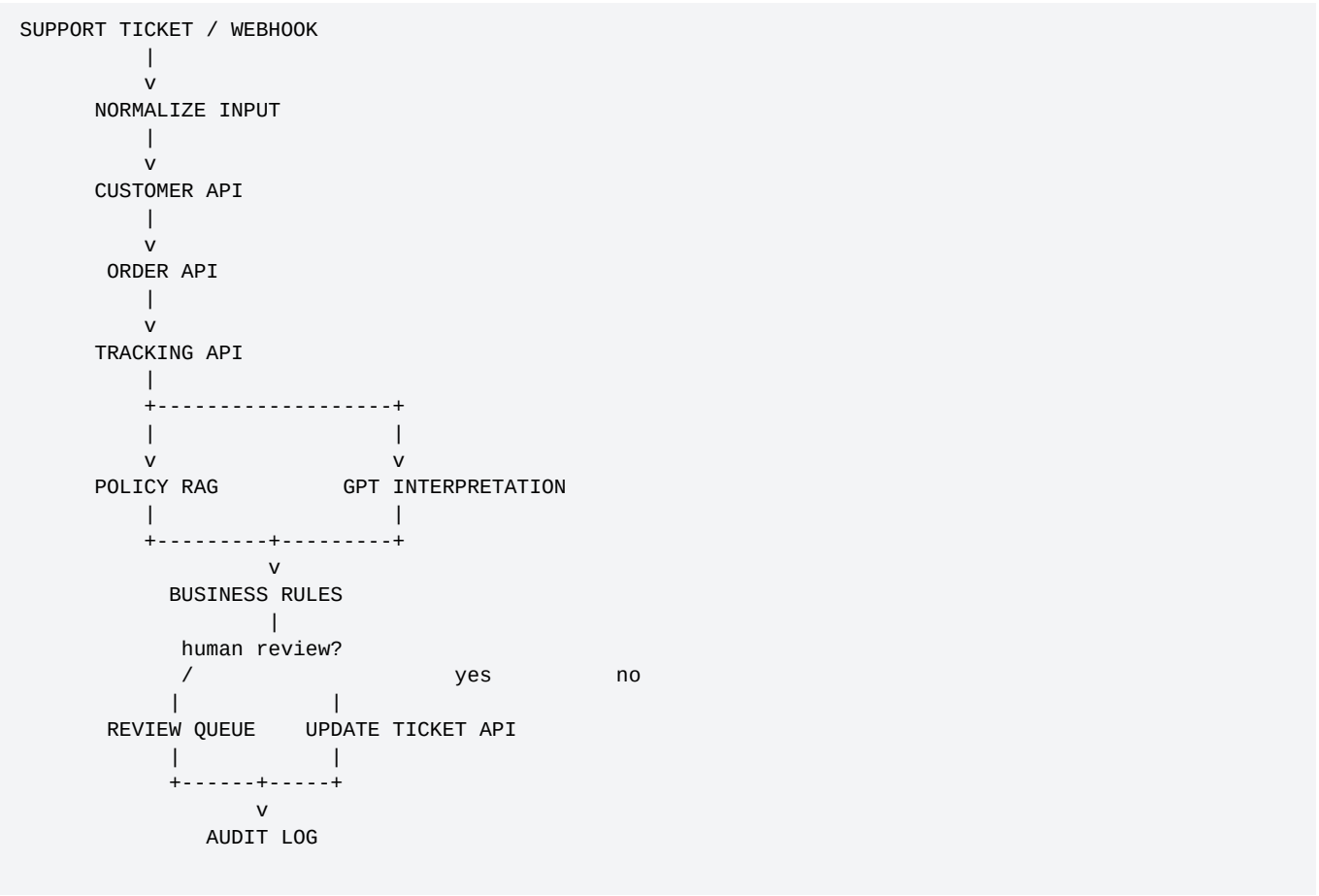
Available Systems

System	Operation
Support API	Get/update ticket
Customer API	Get verified customer profile
Order API	Find order owned by customer
Tracking API	Get current carrier status
RAG knowledge base	Retrieve shipping-delay policy
OpenAI/GPT	Classify and draft response

Target Output

```
{
  "ticket_id": "TKT-5501",
  "customer_id": "C1002",
  "order_id": "ORD-1002",
  "verified_status": "delayed",
  "policy_action": "priority_followup",
  "priority": "high",
  "draft_response": "...",
  "human_review": false
}
```

24. Mini Project Architecture



Why This Architecture Is Strong

- Live facts come from APIs, not memory or model guesses.
- Policy comes from the knowledge base.
- AI handles language interpretation and drafting.
- Deterministic rules set thresholds and approvals.
- Write action happens only after validation.
- Every external step can be logged and retried independently.

25. Mini Project - Node-by-Node Build

#	n8n Node	Purpose
1	Webhook / Trigger	Receive ticket event
2	Edit Fields	Normalize ticket ID, customer email, message
3	HTTP Request - Customer	Resolve trusted customer ID
4	IF	Customer found?
5	HTTP Request - Orders	Get relevant order
6	IF	Order ownership/status valid?
7	HTTP Request - Tracking	Retrieve live shipment status
8	RAG / Vector Search	Retrieve delayed-shipment policy
9	OpenAI / LLM Chain	Classify issue and draft response from verified context
10	Structured Output Parser	Validate AI result
11	IF / Switch	Apply priority and human-review rules
12	HTTP Request - Support	Update ticket safely
13	Log / Database	Store audit record
14	Error workflow	Handle failed integration steps

26. Mini Project - Input Normalization

```
Incoming webhook:
{
  "ticket": {
    "id": 5501,
    "requester": {"email": "sarah@example.com"},
    "message": "My package is 3 days late. Order ORD-1002."
  }
}

Normalize to:
{
  "ticket_id": "TKT-5501",
  "customer_email": "sarah@example.com",
  "message": "My package is 3 days late. Order ORD-1002."
}
```

Customer Lookup Request

```
GET https://api.example.com/v1/customers
Query:
email = {{$json.customer_email}}

Authentication:
Stored n8n credential
```

Validate Customer

```
IF customer not found:
  route to manual review
  do not guess customer_id
```

27. Mini Project - Order and Tracking Calls

Order Search

```
GET /v1/orders
customer_id = {{$json.customer_id}}
order_id    = {{$json.extracted_order_id}}
```

Ownership Rule

```
IF returned_order.customer_id != authenticated_customer_id:
    STOP
    create security/manual-review event
```

Tracking Call

```
GET /v1/tracking/{{$json.tracking_id}}

Expected normalized result:
{
  "tracking_status": "delayed",
  "estimated_delivery": null,
  "last_event_at": "2026-08-18T22:10:00Z"
}
```

KEY TAKEAWAY

The workflow should carry trusted customer and order IDs from system responses, not from the AI layer.

28. Mini Project - AI Input and Structured Output

Only Send Necessary Verified Context

```
CUSTOMER MESSAGE:
{{$json.message}}

VERIFIED ORDER:
status={{$json.order_status}}

VERIFIED TRACKING:
status={{$json.tracking_status}}
eta={{$json.estimated_delivery}}

POLICY EXCERPT:
{{$json.policy_text}}

TASK:
Classify urgency and draft a customer reply.
Do not invent facts outside the supplied context.
```

Structured AI Output

```
{
  "issue_type": "shipping_delay",
  "urgency": "high",
  "summary": "Shipment is delayed with no confirmed ETA.",
  "draft_response": "...",
  "requires_human_review": false
}
```

Deterministic Rule After AI

```
IF tracking_status == "delayed"
AND estimated_delivery is empty
AND customer_deadline <= 1 day:
    priority = "high"
```

29. Mini Project - Safe Ticket Update

Before Write

11. Confirm ticket_id exists.
12. Confirm customer/order ownership checks passed.
13. Confirm AI output schema is valid.
14. Compute priority with deterministic rules.
15. Check human-review policy.
16. Create idempotency/deduplication key if API supports it.

PATCH Ticket

```
PATCH /v1/tickets/TKT-5501
{
  "priority": "high",
  "internal_summary": "Shipment delayed; no carrier ETA.",
  "draft_response": "...",
  "automation_run_id": "run_20260819_001"
}
```

After Write

- Validate status code and response ticket ID.
- Store external request/response metadata without secrets.
- Record workflow execution ID and timestamp.
- If write fails after uncertain server response, verify ticket state before retrying.

30. Mini Project - Failure Matrix

Failure	Workflow Response
Customer API timeout	bounded retry, then manual queue
Customer not found	manual review; no AI guess
Order not found	ask/support queue
Ownership mismatch	security stop
Tracking 429	Wait + retry within policy
Tracking 503	bounded retry then fallback
Policy retrieval empty	do not invent policy; escalate
GPT invalid output	retry once with parser/fallback; then review
Ticket API timeout on PATCH	GET ticket state before repeating write
Duplicate webhook	deduplicate using event/run key

Test Cases

- Normal in-transit shipment.
- Delayed with no ETA.
- Delivered order but customer says missing.
- Invalid order ID.
- Order belongs to another customer.
- Tracking API rate limit.
- Tracking API outage.
- Policy not found.
- Duplicate event delivered twice.
- Ticket update response times out after server may have saved it.

31. API Integration Observability

Log Field	Why It Matters
workflow_execution_id	trace one run end to end
system	which external API
operation	get_order, update_ticket, etc.
status_code	transport result
duration_ms	latency monitoring
retry_count	reliability signal
rate_limit_remaining	capacity signal when available
resource_id	business traceability
success	simple outcome
error_code	group failures by cause

Never Log

- Authorization headers.
- API secrets.
- Passwords.
- Full payment credentials.
- Sensitive personal data unless explicitly required and protected.

Useful Dashboard Metrics

- API success rate by provider.
- p50/p95 latency.
- 429 rate.
- 5xx rate.
- Retry rate.
- Manual escalation rate.
- Duplicate prevention count.
- Cost per successfully automated case.

32. Production Checklist

Area	Checklist
API contract	Endpoint/version documented; response schema known
Credentials	Stored securely; least privilege; production separated
Mapping	Stable normalized schema and system IDs
Pagination	Bounded; checkpoint/delta strategy
Rate limits	Retry/backoff/batch policy
Writes	Idempotency/deduplication and approval rules
Errors	Classified; error workflow configured
Security	No arbitrary URLs; ownership/permission checks
AI	Only necessary data; structured output validated
Observability	Logs/metrics without secrets
Testing	Success, negative, duplicate, rate-limit, outage tests
Operations	Runbook for credential expiry/API changes

KEY TAKEAWAY

Production readiness comes from failure design. The happy-path request is usually the easiest part of API integration.

33. Common Beginner Mistakes

17. Hard-coding API keys in an HTTP Request node or Code node.
18. Calling write endpoints before validating with read endpoints.
19. Assuming a 200 response means the business action is correct.
20. Ignoring pagination and silently processing only the first page.
21. Retrying POST requests without understanding duplicate side effects.
22. Using one vendor ID as if it were a universal customer ID.
23. Sending complete API responses to GPT when only three fields are needed.
24. Allowing user input to control arbitrary HTTP URLs.
25. Ignoring 429 and running a high-concurrency loop.
26. No timeout or maximum retry count.
27. Putting every API call directly in one giant workflow instead of reusable connectors.
28. Logging credentials or sensitive data while debugging.
29. Failing to test missing data, 404s, 5xx, duplicate events, and timeouts.
30. Using AI to make exact authorization decisions.
31. Not monitoring external API version changes or credential expiration.

34. Classroom Exercises

Exercise 1 - Read an API Contract

Given: GET /customers/{id}, bearer auth, 200/404/429 responses. Students identify method, path parameter, auth, success result, and retry policy.

Exercise 2 - Build a GET Request

Configure an n8n HTTP Request node with a dynamic customer_id and normalize the returned JSON.

Exercise 3 - Build a POST Request

Create a mock support ticket payload, but add validation and a deduplication key first.

Exercise 4 - Pagination

Design cursor pagination and define a maximum page count of 20.

Exercise 5 - Error Routing

Route 400, 401/403, 404, 429, and 5xx into different recovery paths.

Exercise 6 - AI Responsibility

Mark each step as AI, n8n code/rule, API, RAG, or human: intent detection, get order, check $500 threshold, retrieve policy, approve refund, draft response.

35. Day 26 Quiz

1. Which n8n node is the general-purpose connector for REST APIs?
Answer: HTTP Request.

2. What is the difference between a path parameter and query parameter?
Answer: A path parameter identifies a resource in the URL path; a query parameter filters/configures the request.

3. What usually does HTTP 429 mean?
Answer: The API rate limit was exceeded.

4. Should a 400 invalid request normally be retried unchanged?
Answer: No.

5. Why is idempotency important?
Answer: It prevents retries from accidentally duplicating business actions.

6. What should hold API secrets?
Answer: n8n credentials or another secure secret mechanism, not prompts or hard-coded node text.

7. What is pagination?
Answer: Retrieving a result set across multiple API requests/pages.

8. Why normalize API data?
Answer: To give downstream workflows a stable internal schema independent of vendor naming.

9. Which component should verify current shipping status?
Answer: The authoritative tracking/order API.

10. Which component should evaluate a strict amount > $500 rule?
Answer: Deterministic code/n8n logic.

11. Should retries be infinite?
Answer: No, use bounded retries and fallback/escalation.

12. What is the safest response to a 403?
Answer: Stop the action and fix permission/scope; do not blind retry.

13. What should happen if a write request times out and the server may have completed it?
Answer: Verify current resource state before retrying.

14. Is an AI-generated customer ID a trusted identifier?
Answer: No.

15. What is a reusable connector workflow?
Answer: A sub-workflow that encapsulates API auth, request, normalization, and error behavior behind a stable input/output contract.

36. Homework - Build Your Own External API Integration

Homework A - One Read API

32. Choose a safe API or sandbox.
33. Document endpoint, method, auth, parameters, pagination, and rate limit.
34. Build the request in n8n.
35. Normalize the response.
36. Handle 404, 429, and 5xx paths.

Homework B - One Write API

37. Choose a non-destructive sandbox write such as creating a test note/task/ticket.
38. Validate required fields before calling API.
39. Add duplicate prevention/idempotency strategy.
40. Log returned resource ID.
41. Test a timeout/retry scenario safely.

Homework C - Multi-API Workflow

42. API 1 retrieves a record.
43. API 2 enriches or verifies it.
44. GPT performs one language task.
45. Deterministic rule makes one strict decision.
46. API 3 writes a result.
47. Create an audit log and error branch.

37. Homework - Architecture Deliverables

Create a short technical design containing all of the following:

- Business problem statement.
- Workflow architecture diagram.
- API inventory and required scopes.
- Input/output schemas.
- ID mapping strategy.
- Pagination strategy.
- Rate-limit and retry strategy.
- Idempotency/duplicate strategy.
- AI vs deterministic responsibility table.
- Human approval rules.
- Error matrix.
- Test plan with at least 15 cases.
- Operational monitoring fields.

Portfolio Evidence

- Workflow screenshot.
- Sample normalized JSON.
- Example success execution.
- Example handled 429 or 5xx execution.
- Example duplicate prevention.
- Architecture diagram.
- Short README explaining business value and reliability design.

38. Interview and Upwork Preparation

How do you integrate an API that has no native n8n node?
Use the HTTP Request node, configure auth/endpoint/parameters/body, normalize output, and add reliability/error handling.

How do you handle API rate limits?
Respect vendor limits, use bounded retries/backoff, Wait/batching/concurrency controls, and monitor 429 responses.

How do you safely retry writes?
Use idempotency keys or deduplication, verify current resource state after ambiguous failures, and never blindly replay sensitive actions.

How do you protect API credentials?
Use n8n credential storage, least privilege, separate environments, and never expose secrets to AI prompts or logs.

How do you integrate AI with live data?
Use APIs as the source of truth; pass only necessary verified fields to the model; validate structured output; apply deterministic business rules before write actions.

How do you manage large API result sets?
Pagination, date filters/delta sync, item limits, batching, checkpointing, and rate-limit-aware processing.

How do you debug a failing API workflow?
Inspect status code, response body, request parameters, credentials/scopes, execution logs, retry history, and reproduce the smallest request.

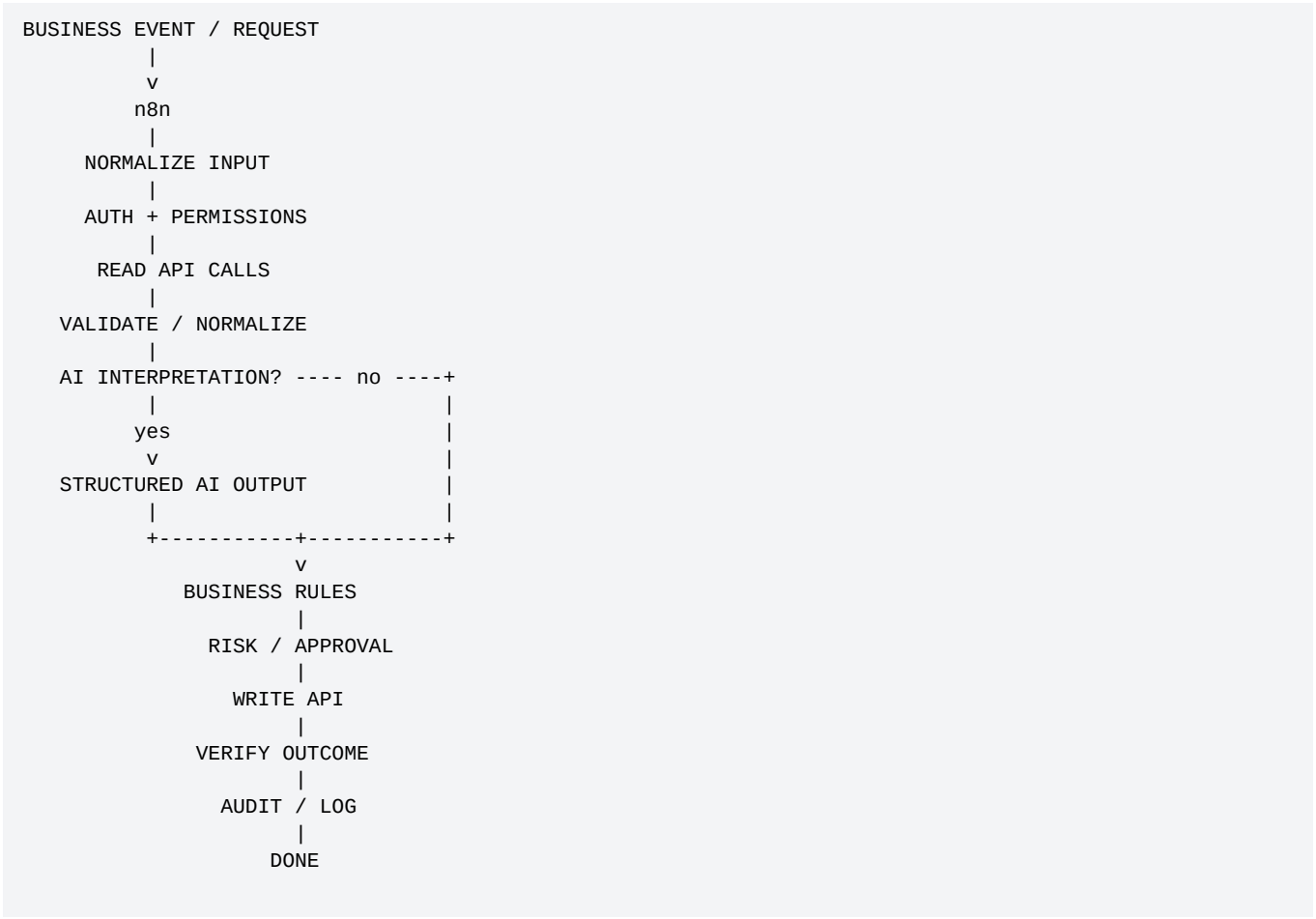
39. Upwork Client Discovery Questions

- Which systems need to exchange data?
- Does each system provide a REST API, webhook, or native n8n node?
- What authentication method and scopes are available?
- Which system is the source of truth for each data field?
- How are customers/orders/contacts mapped across platforms?
- What is the API rate limit and expected daily volume?
- Do endpoints support pagination, webhooks, delta sync, or updated_since filters?
- Which actions create irreversible or financial side effects?
- What actions require human approval?
- What should happen during API downtime?
- How should duplicates be prevented?
- What logs and audit trail are required?
- What sensitive data must not enter the AI layer?
- What are success metrics: time saved, error reduction, SLA, conversion, or support resolution time?

PROFESSIONAL POSITIONING

Do not sell "I can connect APIs." Sell "I can design reliable, secure, observable business integrations that use AI only where AI adds value."

40. Day 26 Final Mental Model



Day 26 Core Formula

FORMULA

Reliable API Automation = Correct Request + Secure Authentication + Data Normalization + Pagination + Rate-Limit Handling + Error Strategy + Idempotent Writes + Validation + Observability

41. Day 26 Completion Checklist

- Can explain REST request anatomy.
- Can configure an n8n HTTP Request node.
- Can use credentials rather than hard-coded secrets.
- Can map path/query/header/body fields.
- Can interpret common HTTP status codes.
- Can normalize vendor responses.
- Can design pagination.
- Can handle 429 and temporary 5xx errors.
- Can explain retries and idempotency.
- Can distinguish read vs write risk.
- Can separate AI interpretation from authoritative API data.
- Can build multi-API workflows.
- Can design reusable API connector sub-workflows.
- Can create failure matrices and test cases.
- Can explain the architecture to a client.

KEY TAKEAWAY

If you can reliably connect n8n to several external systems, normalize their data, handle failures, and control writes, you have moved from "workflow builder" toward "AI Automation Engineer / Architect."

42. Preview - Day 27: Business Automation Patterns

Day 27 will combine GPT, n8n, APIs, RAG, rules, state, and human approvals into reusable business automation patterns.

Pattern	Example
Intake -> classify -> route	Support/email operations
Retrieve -> decide -> act	Order exception handling
Enrich -> score -> update	Sales/CRM automation
Extract -> validate -> approve	Invoice/document operations
Monitor -> detect -> alert	Inventory/operations
Draft -> approve -> publish	Marketing/content workflows
RAG -> answer -> escalate	Knowledge support
Agent -> tools -> human gate	Controlled agentic automation

```
Day 27 Goal:
Stop designing workflows node-by-node.
Start recognizing reusable automation architectures.
```

43. References and Further Reading

Current n8n references used for the time-sensitive implementation details in this lecture:

- n8n - HTTP Request node
- n8n - HTTP Request credentials
- n8n - Handling API rate limits
- n8n - Handle rate limits / Retry On Fail
- n8n - Handle errors gracefully
- n8n - Work with nodes / error settings
- MDN - HTTP overview

Reference note: API and n8n behavior can evolve. For production work, re-check the current vendor and n8n documentation before deployment, especially authentication, limits, pagination, and retry behavior.